

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 2

Programación Dinámica

3 de mayo de 2026

Alejandro, Pablo Martin
98021

Prada, Matias Leonel
99397

Poli Salvador, Ignacio Manuel
110399

1. Consideraciones

En este trabajo practico, vamos a estar resolviendo el problema de Scaloni para definir los días que deba entrenarse y los días que convenga descansar de tal forma de tener la mayor ganancia posible para los posibles N entrenamientos.

Para realizarlo, definimos la siguiente nomenclatura:

- El esfuerzo que demanda el día i -ésimo es e_i , a su vez, esta es la ganancia máxima para ese día.
- La energía por día consecutivo disponible en el día i -ésimo viene definida por s_i
- Para cada día s_i (con $1 \leq i \leq N$) : $s_1 \geq s_2 \geq s_3 \geq \dots \geq s_N$.
- Tenemos N días de de esfuerzo al igual que tenemos N días de entrenamiento consecutivos.

Ejemplo de Resolución

2. Análisis del Problema

El problema presenta una serie de decisiones secuenciales donde cada día i el equipo técnico debe elegir entre dos opciones mutuamente excluyentes: **entrenar** o **descansar**.

Sabemos que la ganancia de un entrenamiento no es estática, sino que depende directamente de la fatiga acumulada del plantel. Podemos expresar la ganancia de entrenar un día específico j , habiendo entrenado ininterrumpidamente desde un día i (es decir, llevando $j - i + 1$ días de fatiga), mediante la siguiente expresión matemática:

$$g(i, j) = \text{mín}(e_j, s_{j-i+1}) \quad (1)$$

en donde e_j es la expectativa de esfuerzo del día j , y s representa el arreglo de energía disponible del equipo, el cual decrece a medida que aumentan los días consecutivos de entrenamiento.

2.1. Matriz de Ganancias

Para evitar recalcular las sumatorias de energía repetidas veces y aislar el concepto de “días consecutivos”, introducimos una matriz de ganancias $\text{Ganancias}(i, j)$. Esta estructura almacena la ganancia total obtenida si el equipo entrena de forma ininterrumpida desde el día i hasta el día j .

Debido a que la ganancia es acumulativa, podemos definir la construcción de esta matriz de manera iterativa:

$$\text{Ganancias}(i, j) = \begin{cases} 0 & \text{si } j < i \\ \text{Ganancias}(i, j-1) + \text{mín}(e_j, s_{j-i+1}) & \text{si } j \geq i \end{cases} \quad (2)$$

Esta precomputación resulta en una matriz triangular superior que se puede construir en tiempo $\mathcal{O}(n^2)$, y nos permite consultar la ganancia de cualquier racha de entrenamientos en tiempo constante $\mathcal{O}(1)$.

2.2. Ejemplo de la Matriz de Ganancias

Para ilustrar la construcción y el significado de la matriz $\text{Ganancias}(i, j)$, consideremos un caso de ejemplo con $n = 4$ días, donde los arreglos de esfuerzo esperado y energía disponible son:

- Esfuerzo (e): [10, 5, 12, 9]
- Energía (s): [15, 7, 5, 2]

En la estructura de esta matriz:

- Las **filas** (i) representan el **día de inicio** del bloque de entrenamiento ininterrumpido.
- Las **columnas** (j) representan el **día de finalización** de ese mismo bloque.
- Cada **celda** contiene la ganancia total acumulada si el equipo sale a entrenar todos los días en ese rango $[i, j]$.

Calculamos los primeros valores paso a paso aplicando la ecuación (2):

- **Fila 1 (Entrenamientos que empiezan el Día 1):** Arrancamos con la energía al máximo ($s_1 = 15$).

- $j = 1$ (Entrenar solo D1): $\min(e_1, s_1) = \min(10, 15) = 10$
 - $j = 2$ (Entrenar D1 y D2): $\text{Ganancias}(1, 1) + \min(e_2, s_2) = 10 + \min(5, 7) = 10 + 5 = 15$
 - $j = 3$ (Entrenar D1 al D3): $\text{Ganancias}(1, 2) + \min(e_3, s_3) = 15 + \min(12, 5) = 15 + 5 = 20$
- **Fila 2 (Entrenamientos que empiezan el Día 2):** Como asumimos que empezó el Día 2, la fatiga se resetea y usamos s_1 para el primer día de este bloque.
- $j = 2$ (Entrenar solo D2): $\min(e_2, s_1) = \min(5, 15) = 5$
 - $j = 3$ (Entrenar D2 y D3): $\text{Ganancias}(2, 2) + \min(e_3, s_2) = 5 + \min(12, 7) = 5 + 7 = 12$

Al completar todos los cálculos, la matriz resultante queda de la siguiente manera:

Inicio (i) \ Fin (j)	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$i = 1$	10	15	20	22
$i = 2$	-	5	12	17
$i = 3$	-	-	12	19
$i = 4$	-	-	-	9

Cuadro 1: Matriz de Ganancias precomputada para $n = 4$.

Por ejemplo, si observamos la celda en la fila $i = 2$ y columna $j = 4$, el valor 17 representa explícitamente la ganancia obtenida si el equipo decide descansar el día 1, y luego entrena ininterrumpidamente durante los días 2, 3 y 4. Las celdas vacías debajo de la diagonal no son válidas matemáticamente, ya que un bloque de entrenamiento no puede terminar en un día anterior al que empezó.

2.3. Ecuación de Recurrencia (1D)

Con la matriz de ganancias definida, podemos modelar el problema principal utilizando Programación Dinámica. Definimos nuestro arreglo unidimensional de subproblemas $OPT(i)$ como “la ganancia máxima posible considerando únicamente los primeros i días del torneo”.

Para entender la lógica detrás de la ecuación, imaginemos que estamos parados en el día i y queremos evaluar cómo maximizar nuestra ganancia acumulada hasta hoy. En lugar de preguntarnos si hoy entrenamos o descansamos, el algoritmo cambia el enfoque y se pregunta: **¿De qué tamaño será la racha ininterrumpida de entrenamientos que finaliza exactamente hoy?**

Si llamamos k a la longitud de esa racha final de entrenamientos, el algoritmo evalúa exhaustivamente todas las opciones físicamente posibles (desde $k = 1$ hasta $k = i$) y se queda con la que otorgue el mayor puntaje. Al evaluar cada k , el problema se divide en dos escenarios:

- **Escenario de Racha Total ($k = i$):** Significa que la decisión a evaluar es si conviene que el equipo haya entrenado todos los días de corrido desde que empezó el torneo hasta el día de hoy, sin descansar nunca. Como no hay días previos, la ganancia de esta opción es directamente el bloque completo que ya precomputamos: $\text{Ganancias}(1, i)$.
- **Escenario de Racha Parcial ($1 \leq k < i$):** Significa que el equipo entrena los últimos k días consecutivos (desde el día $i - k + 1$ hasta el día i), pero **obligatoriamente descansó el día inmediatamente anterior** (el día $i - k$).

Al asumir que el día $i - k$ fue un día de descanso, sabemos que la fatiga del equipo se reseteó a cero. Por lo tanto, el puntaje total de esta opción será la suma de la ganancia de la nueva racha actual de k días, más la **mejor historia posible** que el equipo haya podido construir en los días previos a ese descanso.

Esa “mejor historia pasada” (que abarca desde el día 1 hasta el día $i - k - 1$) ya fue resuelta por el algoritmo en iteraciones anteriores y está guardada en $OPT(i - k - 1)$.

Al unificar esta lógica matemática, formalizamos la ecuación evaluando de forma independiente la racha total para evitar acceder a índices negativos o inválidos en el arreglo OPT :

$$OPT(i) = \max \begin{cases} \text{Ganancias}(1, i) & \text{(Racha total sin descansos)} \\ \max_{k \in \{1, \dots, i-1\}} \left(OPT(i-k-1) + \text{Ganancias}(i-k+1, i) \right) & \text{(Rachas parciales con descanso previo)} \end{cases}$$

2.4. Ejemplo del Cálculo de Óptimos (OPT)

Continuando con nuestro ejemplo de $n = 4$ días y utilizando la matriz de la Tabla 1, evaluaremos el arreglo de óptimos OPT aplicando la ecuación (3) de forma ascendente (*bottom-up*), recordando que asumimos $OPT(0) = 0$.

- **Día 1** ($i = 1$): La única opción válida es entrenar desde el inicio ($k = 1$).
 - $OPT(1) = \text{Ganancias}(1, 1) = 10$. (*Guardamos* $k = 1$).
- **Día 2** ($i = 2$): Evaluamos los posibles cortes k :
 - Entrenar D1 y D2 ($k = 2$): $\text{Ganancias}(1, 2) = 15$.
 - Descansar D1, entrenar D2 ($k = 1$): $OPT(0) + \text{Ganancias}(2, 2) = 0 + 5 = 5$.
 - El máximo es **15**. (*Guardamos* $k = 2$).
- **Día 3** ($i = 3$): Evaluamos los posibles cortes k :
 - Entrenar D1 al D3 ($k = 3$): $\text{Ganancias}(1, 3) = 20$.
 - Descansar D1, entrenar D2 y D3 ($k = 2$): $OPT(0) + \text{Ganancias}(2, 3) = 0 + 12 = 12$.
 - Descansar D2, entrenar D3 ($k = 1$): $OPT(1) + \text{Ganancias}(3, 3) = 10 + 12 = 22$.
 - El máximo es **22**. (*Guardamos* $k = 1$).
- **Día 4** ($i = 4$): Evaluamos los posibles cortes k :
 - Entrenar D1 al D4 ($k = 4$): $\text{Ganancias}(1, 4) = 22$.
 - Descansar D1, entrenar D2 al D4 ($k = 3$): $OPT(0) + \text{Ganancias}(2, 4) = 0 + 17 = 17$.
 - Descansar D2, entrenar D3 y D4 ($k = 2$): $OPT(1) + \text{Ganancias}(3, 4) = 10 + 19 = 29$.
 - Descansar D3, entrenar D4 ($k = 1$): $OPT(2) + \text{Ganancias}(4, 4) = 15 + 9 = 24$.
 - El máximo absoluto del problema es **29**. (*Guardamos* $k = 2$).

El arreglo final resulta ser $OPT = [10, 15, 22, 29]$. El valor $OPT(4) = 29$ representa la máxima ganancia posible para todo el torneo.

2.5. Complejidad y Reconstrucción

La solución planteada requiere dos fases. En primer lugar, la construcción de la matriz $\text{Ganancias}(i, j)$ toma tiempo $\mathcal{O}(n^2)$ y espacio $\mathcal{O}(n^2)$.

En segundo lugar, el cálculo del arreglo unidimensional OPT de tamaño n requiere iterar i desde 1 hasta n , y en el peor de los casos, el bucle interno de maximización evalúa $i - 1$ cortes posibles. Esto resulta en una sumatoria de Gauss, la cual también se acota en tiempo $\mathcal{O}(n^2)$.

El resultado óptimo del problema se encontrará en la última posición del arreglo, $OPT(n)$.

Para reducir la complejidad de reconstruir el cronograma de entrenamientos (camino óptimo), se va a almacenar el valor del k final para poder reconstruir cada paso en $\mathcal{O}(1)$, mejorando la reconstrucción a $\mathcal{O}(n)$. Basta con realizar un proceso de *backtracking* desde $OPT(n)$, con el último k obtenido, marcando esos k días como entrenamiento, el día anterior como descanso ($i - k$), y saltando recursivamente al estado del día anterior $OPT(i - k - 1)$, así sucesivamente hasta llegar al inicio (top-down).